

期末大作业报告

字段	内容
课程名称	企业级应用软件设计与开发
项目名称	ShortLink Agent：面向短链运营的智能客服与智能编排系统
方向	方向二：企业级应用软件的 Agent 改造
学号	2025303058
姓名	潘晨铎
专业	软件工程
指导教师	戚欣
提交日期	2026 年 6 月 22 日

摘要

本项目选择“企业级应用软件的 Agent 改造”方向，以已有的企业级短链 SaaS 系统为基础进行智能化升级。原系统已经具备用户鉴权、短链分组、短链创建、批量创建、回收站、访问统计、Redis 缓存、Kafka 异步统计和网关路由等能力，但在实际运营中仍存在操作入口分散、业务参数理解成本高、统计结果缺少自动分析、多轮对话上下文难以延续等问题。

针对这些问题，本项目新增“智能短链运营助手”，将自然语言作为新的业务入口。用户可以通过“创建 7 天有效短链”“查看当前分组短链列表”“分析最近 7 天访问情况”等指令完成短链创建、查询和统计分析。Agent 层不直接操作数据库，而是将原有 GroupService、ShortLinkService、ShortLinkStatsService、UrlTitleService 等能力封装为工具，通过意图识别、参数抽取、工具编排和结果总结完成业务闭环，保证智能层与原系统权限校验、缓存预热和异步统计链路保持一致。

在工程设计上，系统引入会话记忆压缩、模型候选调度、三态熔断器和用户问题改写机制。会话记忆用于解决多轮对话中的指代和上下文延续问题；模型降级与 safe-fallback 用于提升大模型调用失败时的系统稳定性；问题改写与意图拆分则将用户自然表达转化为可执行的工具参数。最终，系统形成了“理解目标 - 加载记忆 - 识别意图 - 调用工具 - 生成回复 - 记录轨迹”的智能运营闭环，使传统短链管理平台升级为可解释、可降级、可观测的智能客服式业务系统。

关键词：短链系统；智能客服；Agent；Tool Calling；记忆压缩；三态熔断器；Query Rewrite；可观测性

目录

一、选题背景与设计思想..... 4

 1.1 问题定义..... 4

 1.2 原始系统不足与项目价值..... 4

 1.3 技术路线..... 4

二、Specs 规格文档..... 5

 2.1 Product Spec..... 5

 2.2 Architecture Spec..... 5

 2.3 API Spec..... 5

 2.4 工具规格..... 6

三、系统架构与设计..... 6

 3.1 Agent 交互流程..... 7

 3.2 数据流设计..... 7

四、关键实现与代码展示..... 7

 4.1 Agent 核心循环..... 7

 4.2 工具定义与 Trace..... 8

 4.3 配置文件与模型降级..... 9

五、测试与评估..... 12

 5.1 功能测试..... 12

 5.2 Agent 行为评估与 Benchmark..... 13

 5.3 Demo 截图..... 14

六、系统升级与扩展..... 15

七、课程总结..... 16

 7.1 个人收获..... 16

 7.2 工程思维转变..... 16

 7.3 对课程的建议..... 16

一、选题背景与设计思想

1.1 问题定义

原始短链系统已经具备企业级 CRUD 与统计能力，但运营人员在真实使用中仍需在多个页面之间切换：先查看分组，再创建短链，再查询列表，最后进入统计页面分析 PV、UV、UIP 等指标。对于“帮我给知乎创建一个 7 天有效的短链”“分析默认分组最近 7 天访问情况”这类目标式表达，传统页面只能提供分散表单，不能直接理解用户意图。

1.2 原始系统不足与项目价值

不足	表现	Agent 改造价值
入口割裂	创建、查询、统计散落在不同页面	以自然语言统一触发多步业务流程
参数门槛	用户要理解分组、有效期、描述、URL 白名单等字段	从用户话语中抽取 URL、有效期、分组和描述
统计解释弱	图表展示 PV/UV/UIP，但缺少运营建议	自动总结访问表现并给出下一步动作
长对话缺上下文	“它”“这个分组”需要历史信息	会话窗口 + 摘要压缩，避免不失忆与爆 Token 的冲突
模型不稳定	外部模型可能失败、超时或不可用	候选模型 + 三态熔断 + safe-fallback 兜底

1.3 技术路线

技术路线采用“原系统能力复用 + Agent 聚合层增强”的方式：前端新增智能助手页面；后端 aggregation 模块新增 Agent Controller、Agent Service、Memory Service、Routing Executor 和 CircuitBreakerStore；工具层复用 GroupService、ShortLinkService、ShortLinkStatsService、UrlTitleService 等已有业务服务；模型层预留 OpenAI-compatible、本地兼容模型和 safe-fallback 候选策略。

二、Specs 规格文档

2.1 Product Spec

目标用户是短链运营人员。系统应支持用户通过自然语言完成短链创建、分组查询、短链列表查询、访问统计分析和运营建议生成。核心用户故事包括：用户输入“帮我给 <https://www.zhihu.com> 创建一个 7 天有效的短链”，系统自动识别 URL、有效期和当前分组，调用创建工具并返回新短链；用户输入“查看当前分组短链列表”，系统返回标题、短链地址、原始链接和 PV 概览。

2.2 Architecture Spec

架构上新增 aggregation 智能聚合层，位于 console-vue 与原短链业务服务之间。Agent 不直接操作数据库，而是通过工具调用复用原有服务，保证权限、分组、白名单、缓存和统计链路的一致性。模型调度通过 AgentModelProperties 配置候选 provider、优先级、熔断阈值、冷却时间和 fallback 策略。

2.3 API Spec

接口	方法	用途
/api/short-link/admin/v1/agent/chat	POST	Agent 对话、意图识别、工具编排和自然语言回复
/api/short-link/admin/v1/agent/settings	GET	返回模型 provider、候选模型、熔断状态和记忆配置
/api/short-link/v1/create	POST	复用原短链创建能力
/api/short-link/v1/page	GET	复用原短链分页查询能力
/api/short-link/v1/stats/group	GET	复用分组访问统计能力

2.4 工具规格

工具	复用能力	输入	输出
listGroups	GroupService.groupList	用户登录态	分组名称、gid、短链数量
createShortLink	ShortLinkService.createShortLink	URL、gid、有效期、描述	短链、原始链接、分组、有效期
pageShortLink	ShortLinkService.pageShortLink	gid、分页参数	短链列表和 PV 概览
groupShortLinkStats	ShortLinkStatsService.groupShortLinkStats	gid、时间范围	PV/UV/UIP、趋势与运营建议

三、系统架构与设计

本节按要求重点展示架构、Agent 交互流程和数据流。系统设计强调：智能层不替代原系统，而是把原系统稳定 API 封装为可观测工具。

核心架构图：原始短链系统 + 智能客服/运营 Agent

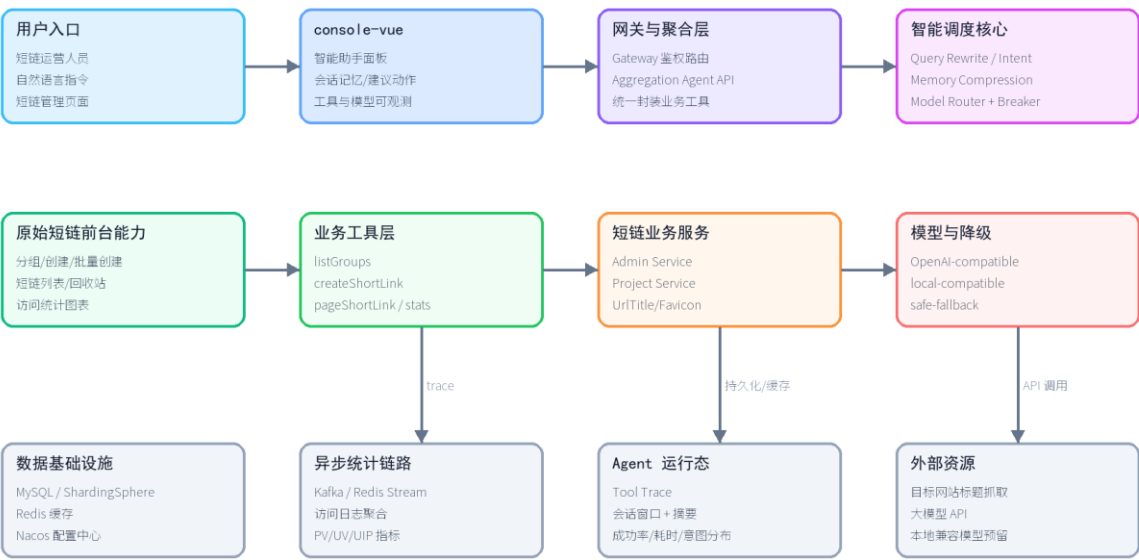


图 1：核心架构图：原始短链系统与智能客服/运营 Agent 的融合架构

3.1 Agent 交互流程

步骤	处理内容	关键产物
1. 接收用户输入	console-vue 将 question、conversationId、当前分组等上下文提交到 Agent API	ShortLinkAgentChatReqDTO
2. 加载记忆	读取最近对话窗口和摘要，恢复“它”“这个分组”等指代	AgentMemorySnapshot
3. 改写与意图识别	归一化有效期、URL、分组等表达，识别 CREATE_SHORT_LINK 等意图	Intent + tool args
4. 工具调用	调用 listGroups/createShortLink/pageShortLink/groupShortLinkStats	AgentToolCallRespDTO
5. 回复与追踪	生成自然语言答复，记录工具耗时、成功率、模型状态和建议动作	ShortLinkAgentChatRespDTO

3.2 数据流设计

短链创建数据流：用户自然语言 -> Agent 参数抽取 -> UriTitleService 补全标题 -> ShortLinkService 创建短链 -> MySQL 保存映射 -> Redis 预热缓存 -> 前端展示结果。访问统计数据流：用户访问短链 -> 跳转服务记录访问事件 -> Kafka/Redis Stream 异步消费 -> 统计表聚合 -> Agent 统计工具读取并生成运营建议。

四、关键实现与代码展示

4.1 Agent 核心循环

```
loadMemory(conversationId)
rewriteQuestion(userQuestion, memory)
intent = resolveIntent(rewrittenQuestion)
candidate = routingExecutor.selectCandidate(intent)
toolResult = invokeBusinessTool(intent, args)
answer = summarize(toolResult, candidate)
appendMemory(userQuestion, answer)
```

return answer + toolTrace + settingsSnapshot

4.2 工具定义与 Trace

每个工具调用均记录 `toolName`、`request`、`responseSummary`、`success`、`durationMs` 和 `message`。这样前端可以展示工具数量、成功率、平均耗时、链路追踪和意图分布，避免 Agent 行为完全黑盒。

实现点	对应文件/模块	说明
Agent 对话入口	ShortLinkAgentController	接收用户问题、 <code>conversationId</code> 和上下文参数
业务编排	ShortLinkAgentServiceImpl	识别意图、解析 URL/有效期/分组、调用业务工具
模型配置	AgentModelProperties	配置 <code>provider</code> 、候选模型、优先级、熔断阈值和兜底策略
三态熔断	AgentCircuitBreakerStore	记录模型健康状态并控制 CLOSED/OPEN/HALF_OPEN 转换
会话记忆	InMemoryAgentMemoryService	维护会话窗口、助手回复和摘要快照
前端面板	console-vue/src/views/agent/AgentIndex.vue	展示聊天、建议动作、图表、模型状态和工具 <code>trace</code>

4.3 配置文件与模型降级

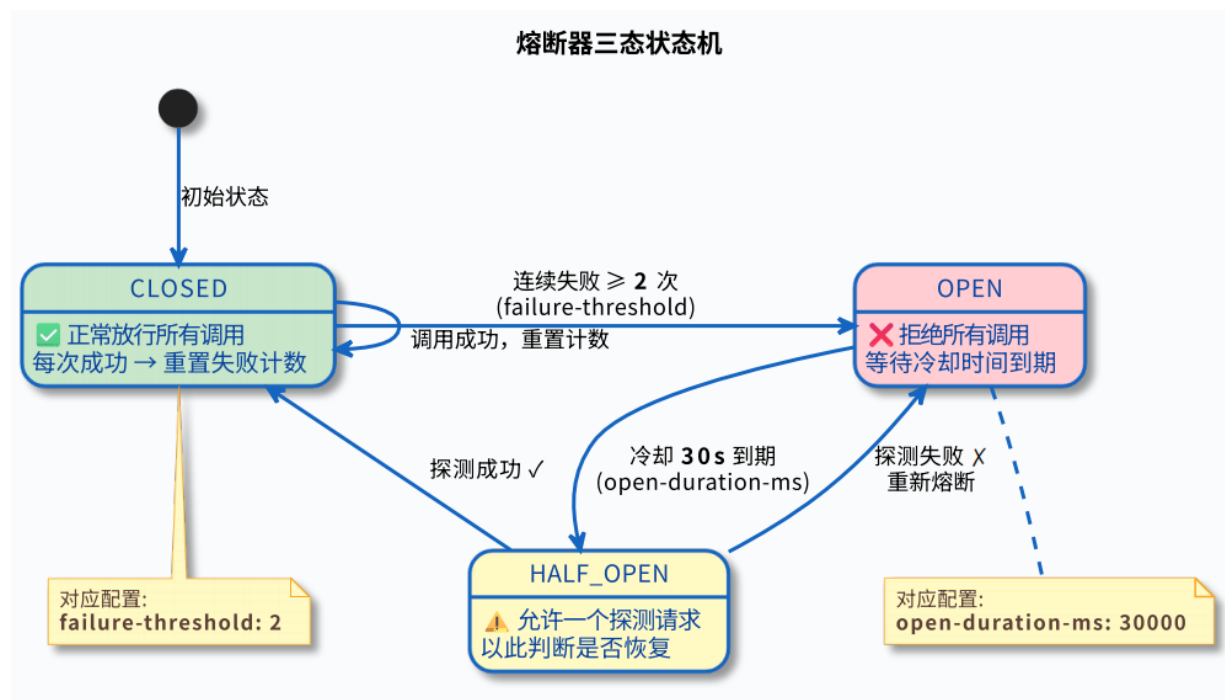


图 2: 三态熔断器状态转换: CLOSED、OPEN、HALF_OPEN

在多模型调度场景中，系统引入了 ModelHealthStore（三态熔断器）和 ModelRoutingExecutor（故障转移执行器）来保障模型调用的高可用与低延迟。其中，ModelHealthStore 底层基于 ConcurrentHashMap 的原子操作来维护每个模型的健康状态，从而在不阻塞全局并发调用的前提下，有效避免了单一模型故障导致的用户请求大面积超时。

该熔断器实现了 CLOSED（正常）、OPEN（熔断）和 HALF_OPEN（半开）三种状态的动态流转。在 CLOSED 状态下，如果某个模型连续调用失败次数达到设定阈值（默认 2 次），系统会将其标记为 OPEN 状态并开启冷却倒计时（默认 30 秒），在此期间所有发往该模型的请求都会被直接拒绝并跳过。冷却时间到期后，模型进入 HALF_OPEN 状态，此时系统仅允许放行一个真实的探测请求：若探测成功，模型恢复为 CLOSED 状态；若探测再次失败，则一票否决重新退回 OPEN 状态并重新计时。

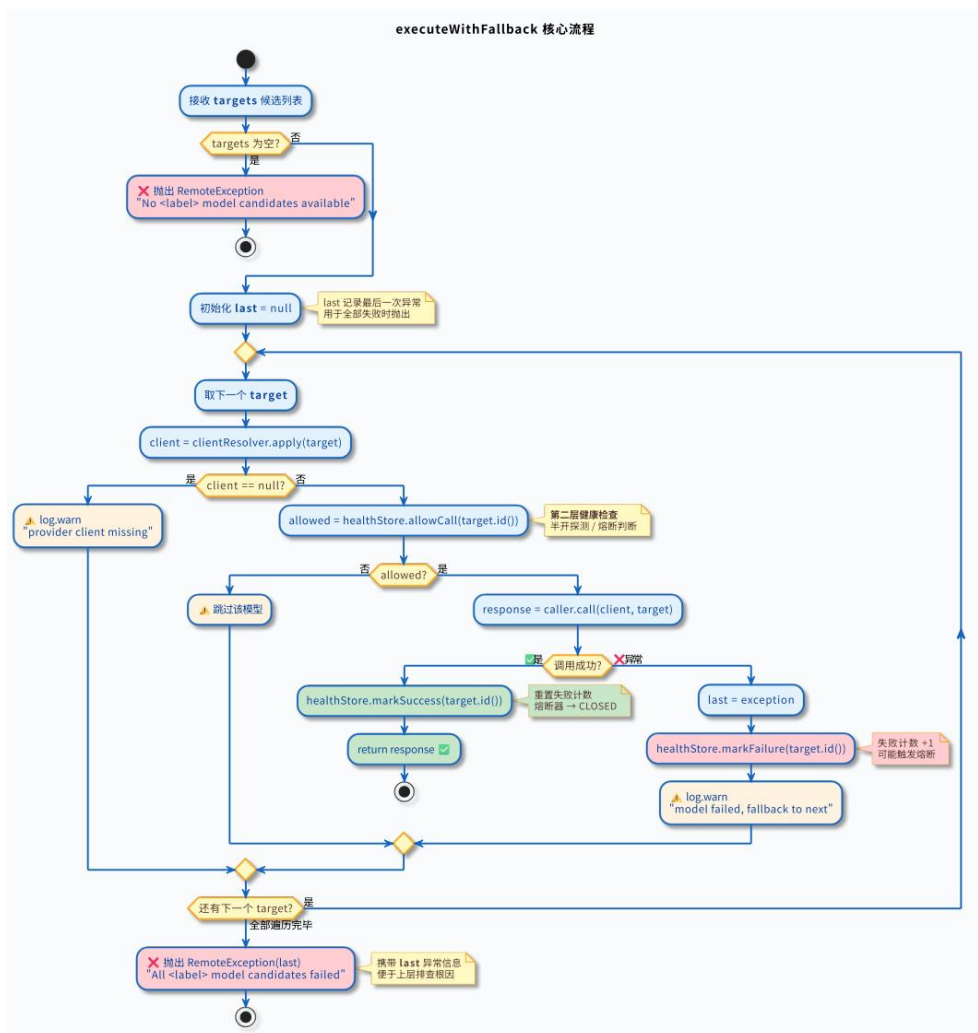


图 3：调用链路中熔断流程

在具体的执行层面，ModelRoutingExecutor 提供了一套纯粹的故障转移逻辑。它会按照优先级遍历模型候选列表，逐一查找客户端实例并进行熔断状态检查。如果当前调用的模型发生异常，执行器会自动捕获错误、通知熔断器记录失败（markFailure），并无缝切换到下一个备选模型继续尝试，直至成功后记录恢复（markSuccess）。得益于泛型与函数式接口（ModelCaller<C, T>）的设计，这套自动降级与容错机制被完美解耦，使得 Chat、Embedding 和 Rerank 等不同能力均能共用同一套高可用逻辑，且对业务调用方完全透明。

4.4 持久化机制

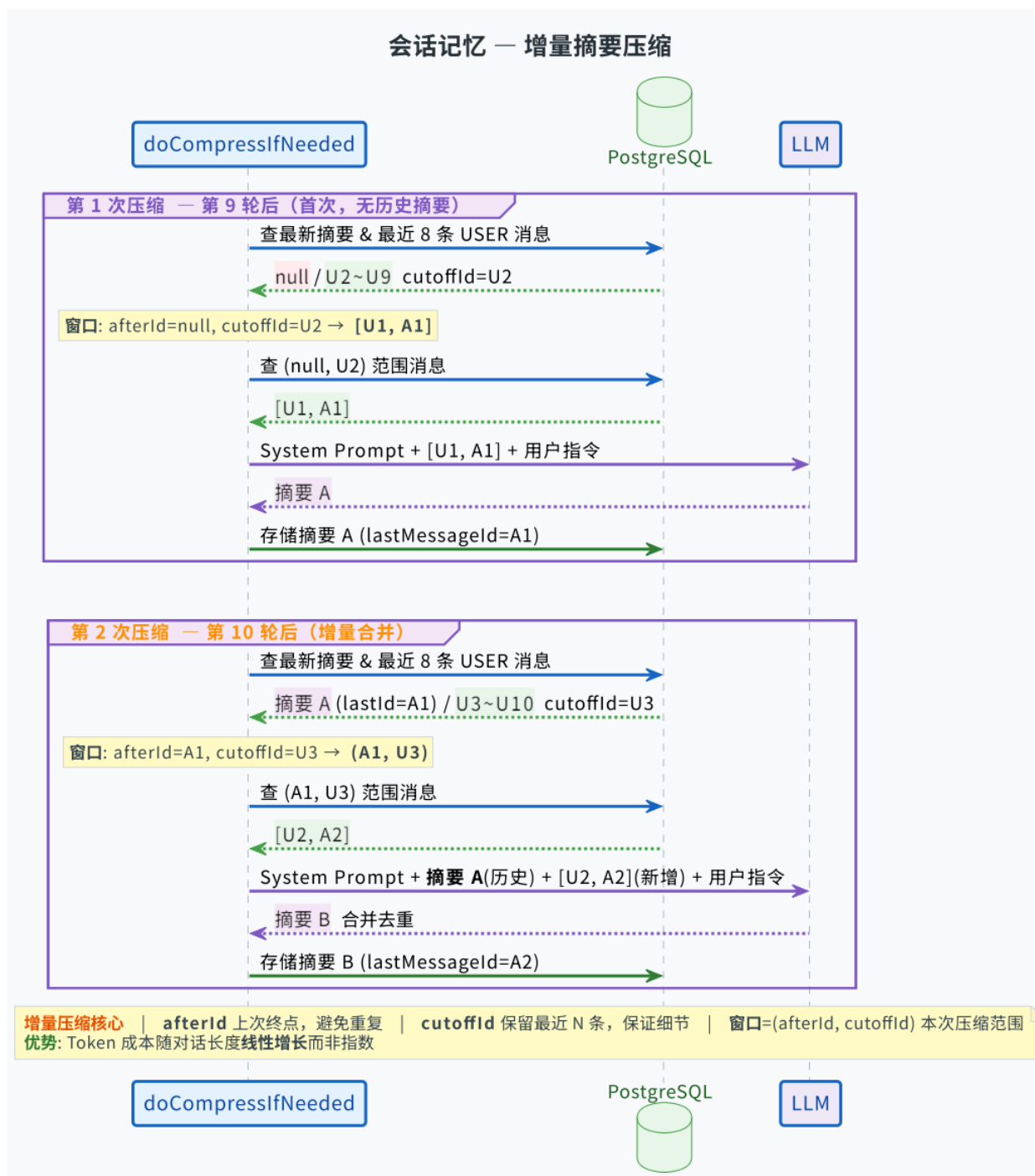


图 4：摘要压缩流程图

针对大模型长对话中 Token 容易超限以及直接丢弃早期对话会导致关键约束丢失的问题，系统采用了一种“滑动窗口+摘要压缩”的混合记忆策略。该策略会完整保留最近

N 轮对话的原文以提供详细上下文，同时利用大模型将更早滑出窗口的历史对话压缩成一段简短的摘要。这段摘要会被加上“对话摘要”的前缀，并作为 **SYSTEM** 消息注入到历史列表的头部，从而在占用极少 **Token** 的情况下，成功保留了用户在早期提出的预算、数量等关键约束条件。

为了保障系统的响应速度，摘要压缩逻辑运行在独立的线程池中异步执行，完全不阻塞用户当前的对话请求。压缩的触发需要同时满足三个条件：全局功能开关已开启、当前消息角色为 **ASSISTANT**、以及对话轮数达到了预设的触发阈值。在执行压缩时，系统巧妙地利用了 `lastMessageId` 作为水位线（类似于书签），明确界定了上次压缩的终点与当前滑动窗口起点之间的消息区间，确保每段历史消息只会被压缩一次。为了进一步控制大模型调用的成本，系统还可以通过设定批处理参数，攒够多轮滑出窗口的消息后再统一合并压缩。

在生成摘要的具体环节，系统通过 **Prompt** 严格约束大模型“只记讨论话题和约束条件，绝不记录具体答案”。这种设计避免了将过期的历史答案固化在摘要中，从而防止与当前实时检索到的最新知识库内容发生冲突。每次执行时，大模型会将原有的历史摘要与新滑出窗口的对话消息放在一起进行合并与去重，生成一份更新后的摘要存入数据库。此外，系统还引入了 **Redis** 分布式锁机制，以防止在多实例集群部署环境下出现并发重复压缩的问题，并且设计了完善的异常降级策略，即使压缩或锁获取失败，也不会影响用户正常的主对话流程。

五、测试与评估

5.1 功能测试

测试项	验证方式	预期结果
后端模块编译	<code>mvn -pl aggregation -am -DskipTests compile</code>	Agent 聚合模块与依赖模块可编译
前端构建	<code>console-vue</code> 执行 <code>npm run build</code>	Agent 页面、图表和路由可打包
创建短链	输入“帮我给 https://www.zhihu.com 创建一个 7 天有效的短链”	返回短链、原始链接、分组和工具 <code>trace</code>
查询列表	输入“查看当前分组短链列表”	返回当前分组短链概览和 PV 数据

模型降级	禁用或模拟候选模型失败	主策略熔断后切换 fallback ，不影响页面可用
记忆展示	连续多轮对话	右侧会话记忆持续记录助手、用户和摘要信息

5.2 Agent 行为评估与 Benchmark

指标	评估方式	意义
意图识别准确率	构造创建、查询、统计、闲聊等样例，检查工具选择是否正确	衡量 Agent 是否理解用户目标
工具调用成功率	统计 <code>success=true</code> 的工具调用占比	衡量业务闭环稳定性
平均工具耗时	记录 <code>durationMs</code> 并在前端展示	衡量用户等待体验
降级可用性	模拟主模型失败，检查 fallback 是否响应	衡量生产级韧性
记忆有效性	多轮追问“它”“这个分组”，检查是否能恢复对象	衡量上下文管理能力

5.3 Demo 截图

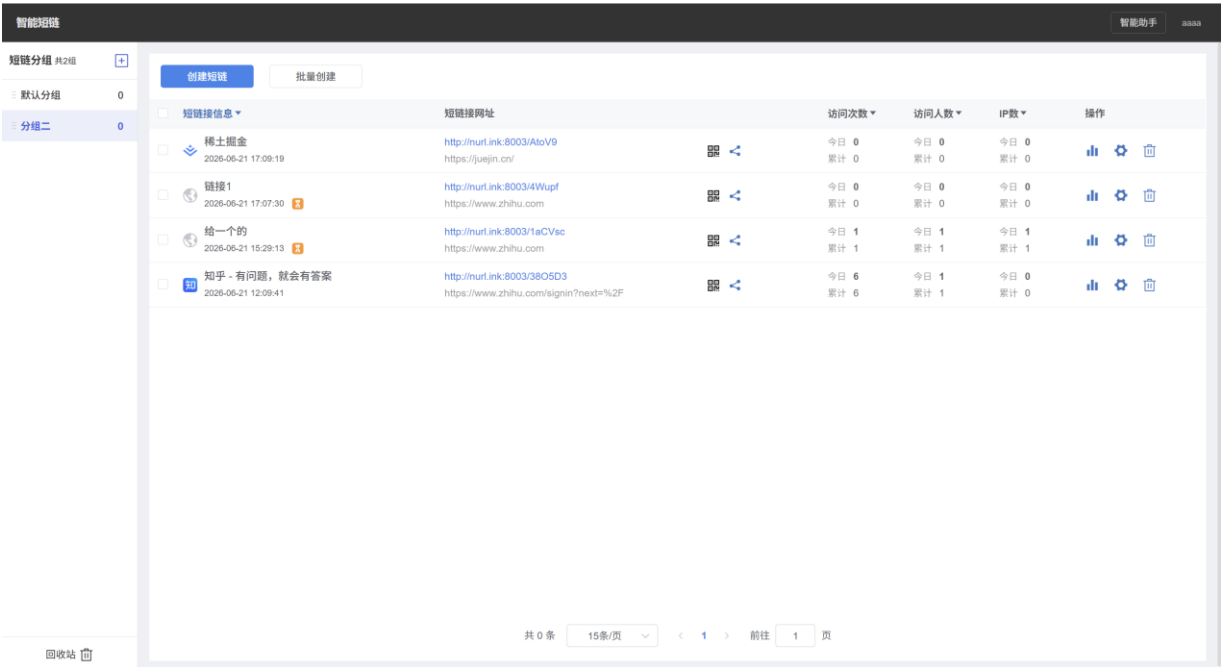


图 5：原始短链列表页面。

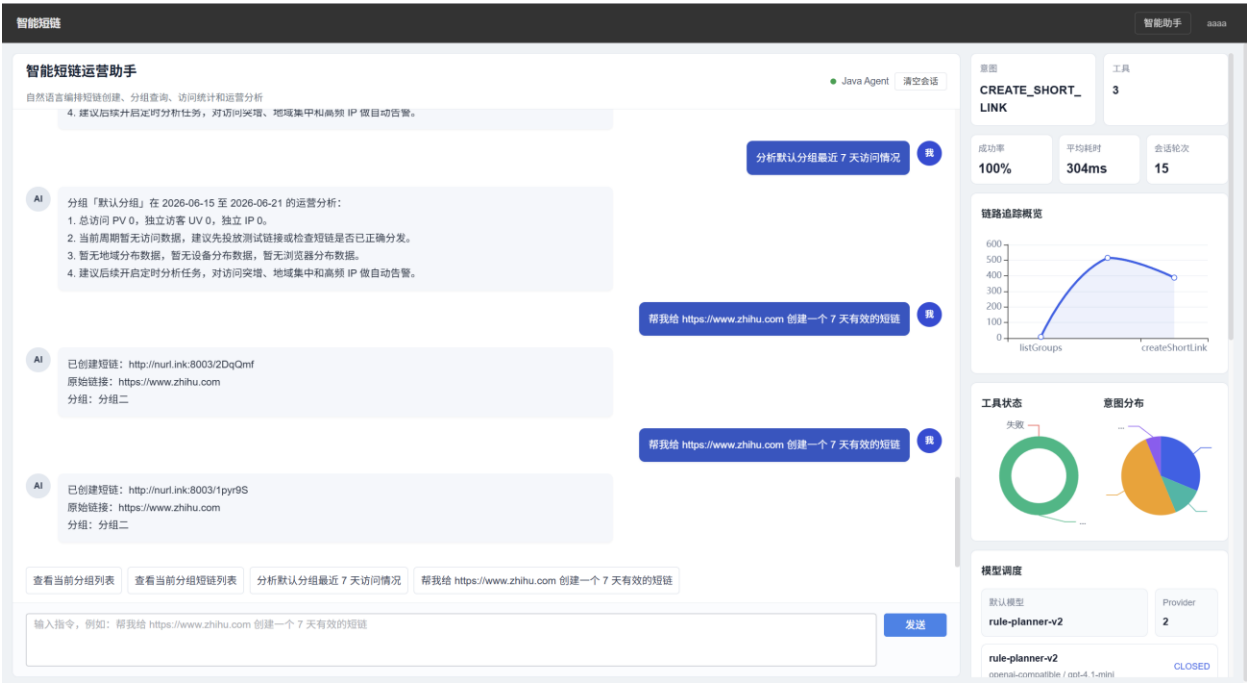


图 6：系统运行截图：智能助手页面与工具可观测面板。



图 7：模型调度、会话记忆与建议动作面板。

六、系统升级与扩展

方向	下一阶段计划	价值
LLMOps Trace	持久化 Tool Trace、意图、耗时、失败原因	形成可评估、可回 放、可优化的数据集
运营预警	定时分析访问突增、高频 IP、异常地域	从问答助手升级为主动运营助手
一键部署	补齐 Docker Compose 启动 MySQL、Redis、Nacos、Kafka 和各服务	降低演示和生产部署成本

AI 能力演进路径为：规则意图识别 -> LLM 工具调用 -> 多工具规划 -> 自动运营报告 -> 主动异常预警。这个路径保留当前系统的稳定性，同时逐步提高智能化程度。

七、课程总结

7.1 个人收获

本项目最重要的收获是理解了企业级 Agent 改造不是简单增加聊天框，而是要围绕原业务系统建立“理解目标 - 选择工具 - 执行业务 - 追踪结果 - 压缩记忆 - 降级恢复”的完整工程闭环。传统 CRUD 更关注单接口正确性，而 Agentic AI 更关注上下文管理、工具边界、状态机、可观测性和失败恢复。

7.2 工程思维转变

在工程设计上，不能让模型直接绕过原业务服务，更不能让智能层直接写数据库。更合理的做法是把已有业务能力封装为工具，让 Agent 做编排，让原系统继续负责权限、校验、缓存、统计和持久化。这样既能利用 AI 提升效率，也能保持企业级系统的可靠性。

7.3 对课程的建议

建议后续课程继续强化 Spec、架构图、Demo 链路和评估指标之间的对应关系，并增加一些生产级 Agent 案例，例如记忆压缩、模型降级、工具可观测和 RAG 评估。这样可以帮助学生从“能跑的 Demo”进一步走向“可维护、可解释、可扩展的工程系统”。